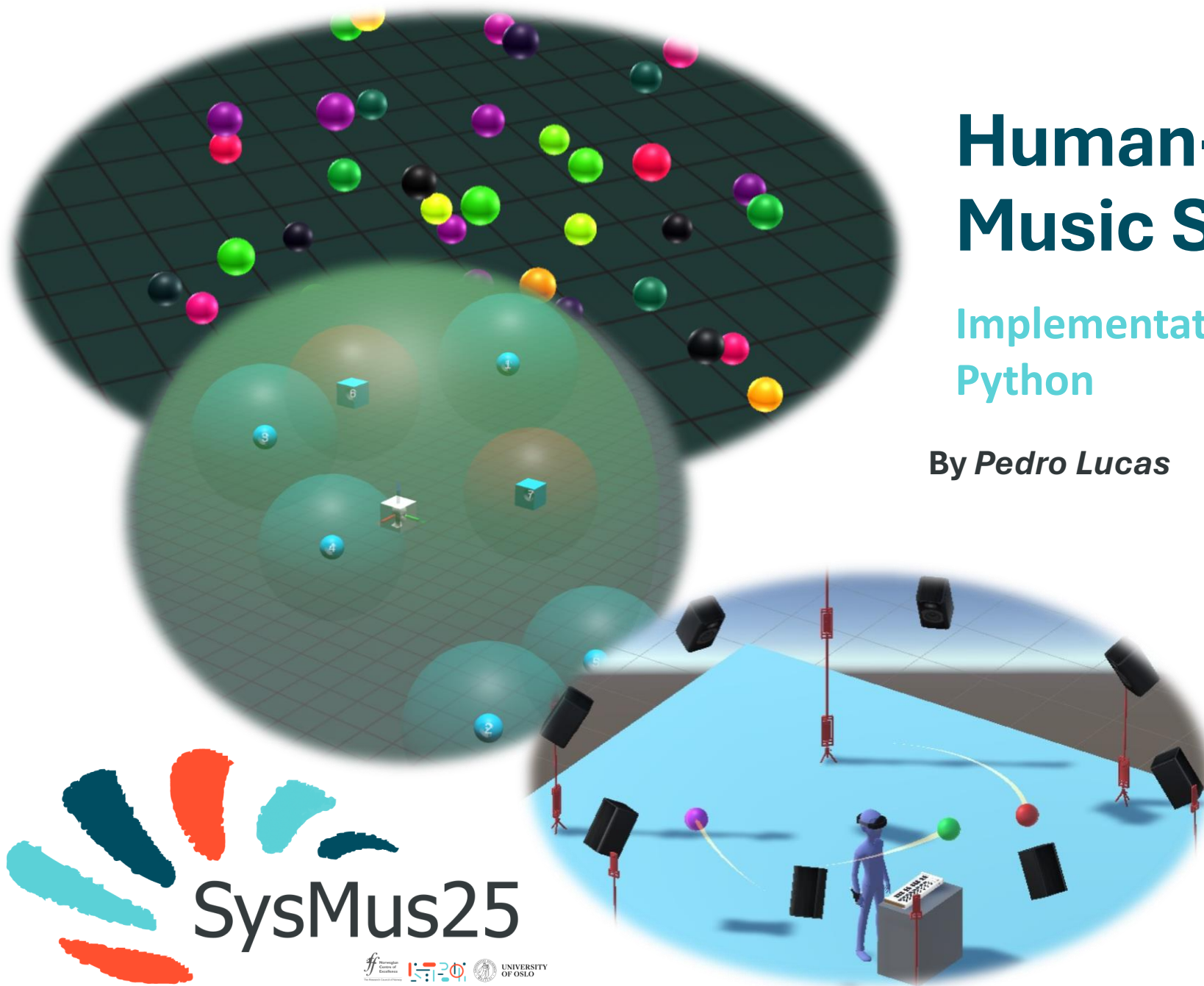


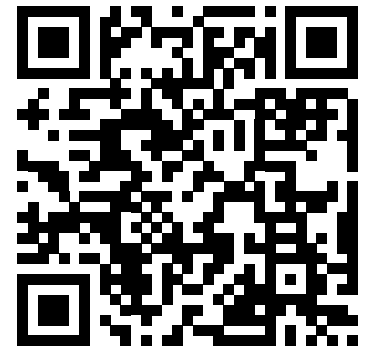
# Human-Swarm Interactive Music Systems

Implementation and data analysis using  
Python

By *Pedro Lucas*



GitHub  
Repository



<https://rb.gy/p8g4jx>

# AGENDA

- **Concepts**
  - *Multi-Agent Systems*
  - *Swarm Intelligence.*
  - *Human-Swarm Interactive Music Systems*
- ***The HS-ims app***
  - *What is it, and how does it work?*
  - *OSC Messages.*
- ***Implementation example in Python***
  - *Description*
  - *Building blocks* (the most technical part)
- ***Data Analysis***
  - *Data Collection using the HS-ims app*
  - *Data Analysis in Python using a Jupyter Notebook*

# Multi-Agent Systems

## Agent

An agent is a computer system that is situated in some **environment**, and that is capable of autonomous **action** in this environment in order to meet its delegated **objectives**. [1]

### Capabilities of an intelligent agent:

- **Reactivity** - Respond to changes in the environment
- **Proactiveness** - Initiate goal directed behaviour on their own
- **Social ability** - Interact with other agents in order to satisfy Goals.  
(cooperation, coordination, negotiation)

# Multi-Agent Systems

## Multi-Agent System

Multi-agent systems are systems composed of multiple **interacting** computing elements, known as agents.[1]

### Types of agents according to design objectives:

- **Reactive (or simple)** - Agents can produce emergent properties. Usually modelled by *Swarm Intelligence (SI)*
- **Interactive (or intelligent)** - Agents engage in social activities like cooperation, coordination, negotiation, etc. Usually described using *Game Theory (GT)*

# Swarm Intelligence

Swarm intelligence is the **emergent** collective intelligence of groups of simple agents, typically based on the social insect metaphor [1].

## The social insect metaphor emphasizes:

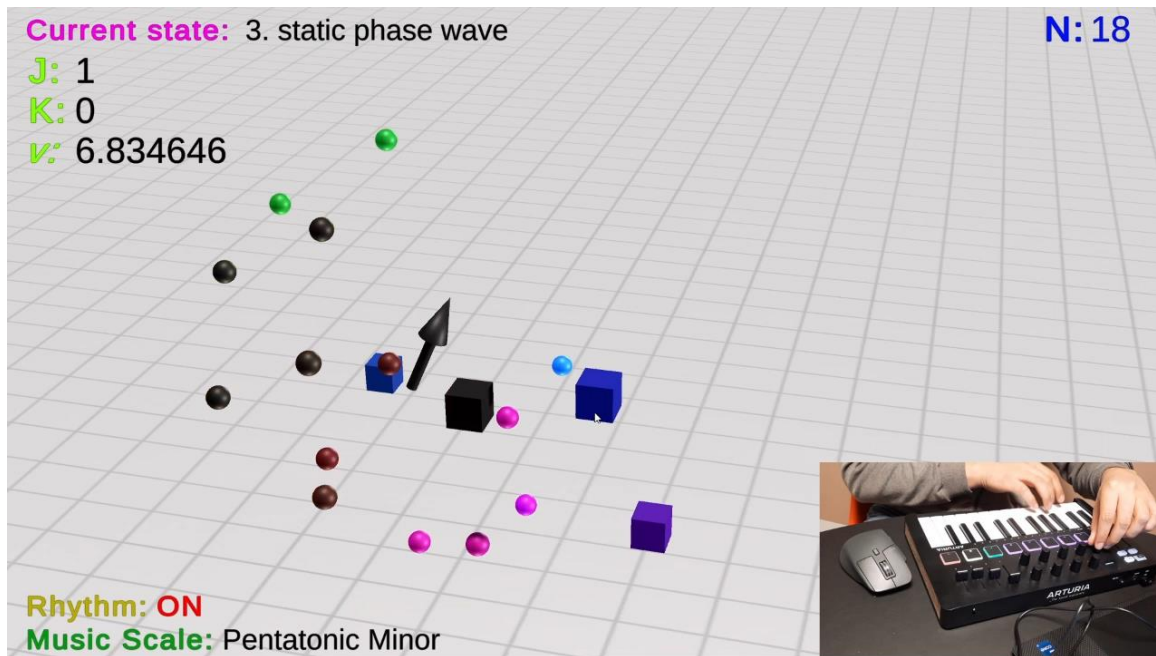
- Autonomy
- Distributedness
- Simple agents (**reactive**)
- Emergence (Properties from the many simple interactions)
  - **Stigmergy** (Local indirect communication)
  - **Self-organization** (Use of local information for global effects)



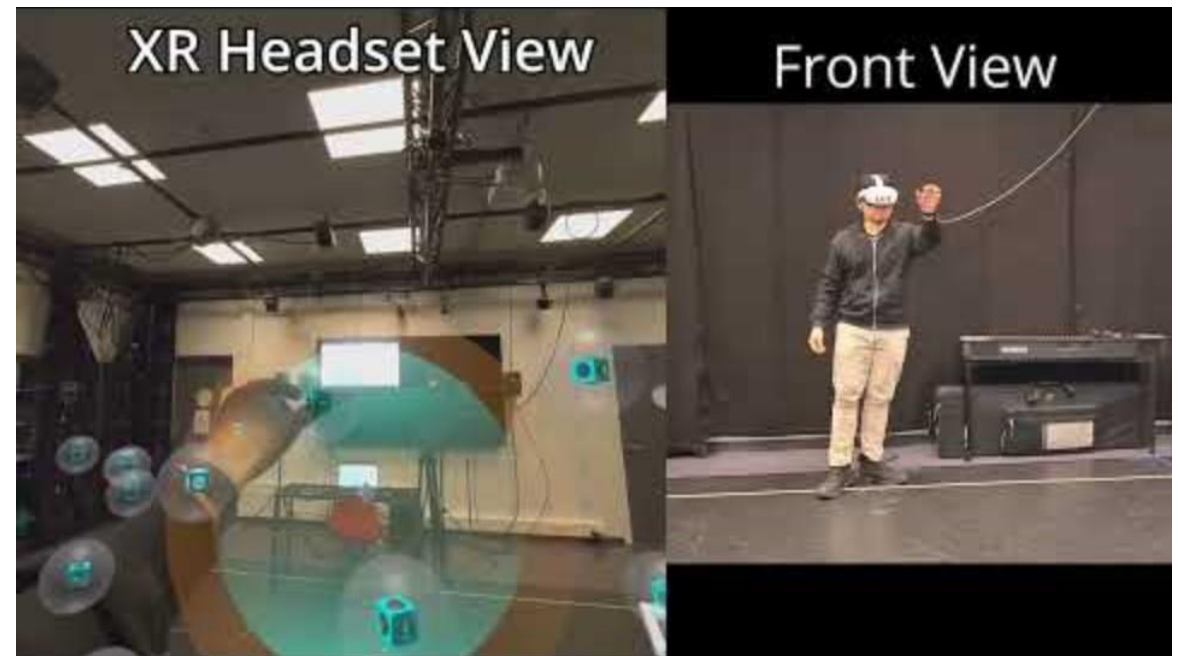


# Human-Swarm Interactive Music System

A Human-Swarm Interactive Music System (IMS) is an improvisational system that allows users to interact with a swarm of **self-organized** artificial agents exhibiting **emergence** in a sound and music context [1].



Swarmalators

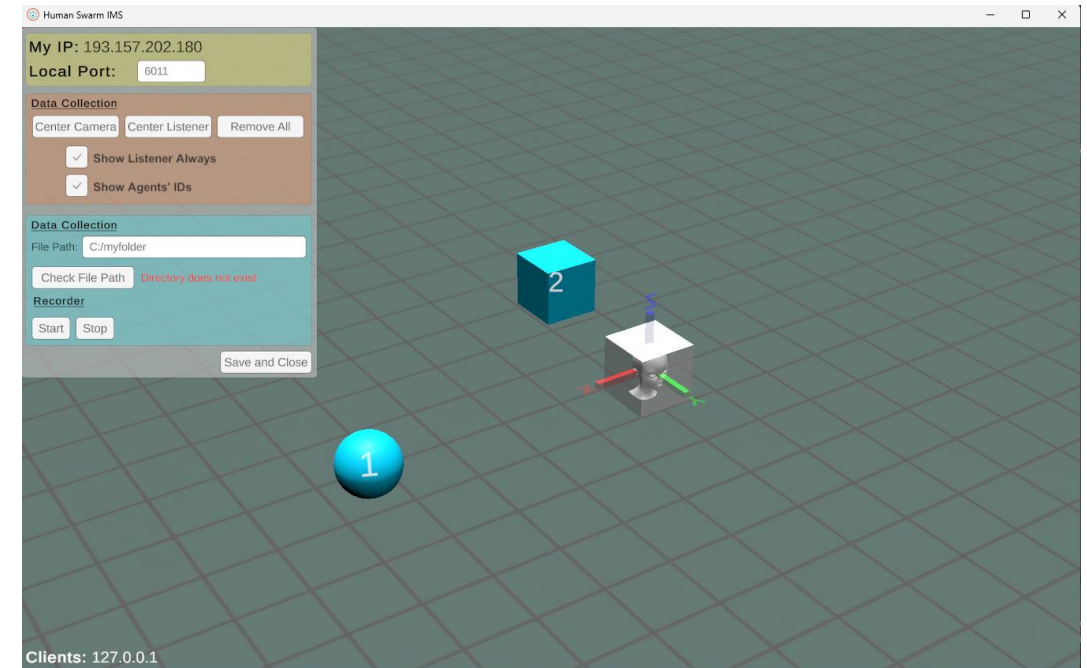


Self-Assembly Agents

[1] Lucas, P., & Glette, K. (2023). Human-Swarm Interactive Music Systems: Design, Algorithms, Technologies, and Evaluation. Proceedings of the 16th International Symposium on Computer Music Multidisciplinary Research, 347–358.

# The Human-Swarm Interactive Music System (HS-ims) app

The app is intended for multi-agent systems with a user interaction focus, especially for sound and music applications. It only supports Windows and Mac OS.



Executables: [https://github.com/pedro-lucas-bravo/human\\_swarm\\_examples/releases/tag/v1.0.0](https://github.com/pedro-lucas-bravo/human_swarm_examples/releases/tag/v1.0.0)

GitHub Repository with examples using the app:

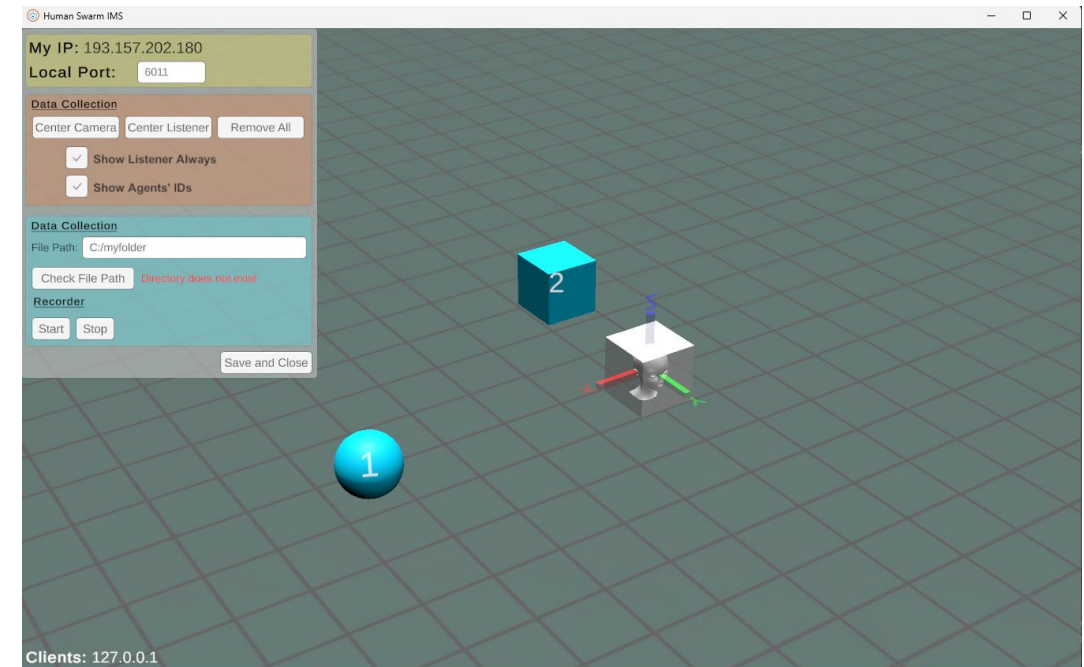
[https://github.com/pedro-lucas-bravo/human\\_swarm\\_examples](https://github.com/pedro-lucas-bravo/human_swarm_examples)

# HS-ims app

Main purposes:

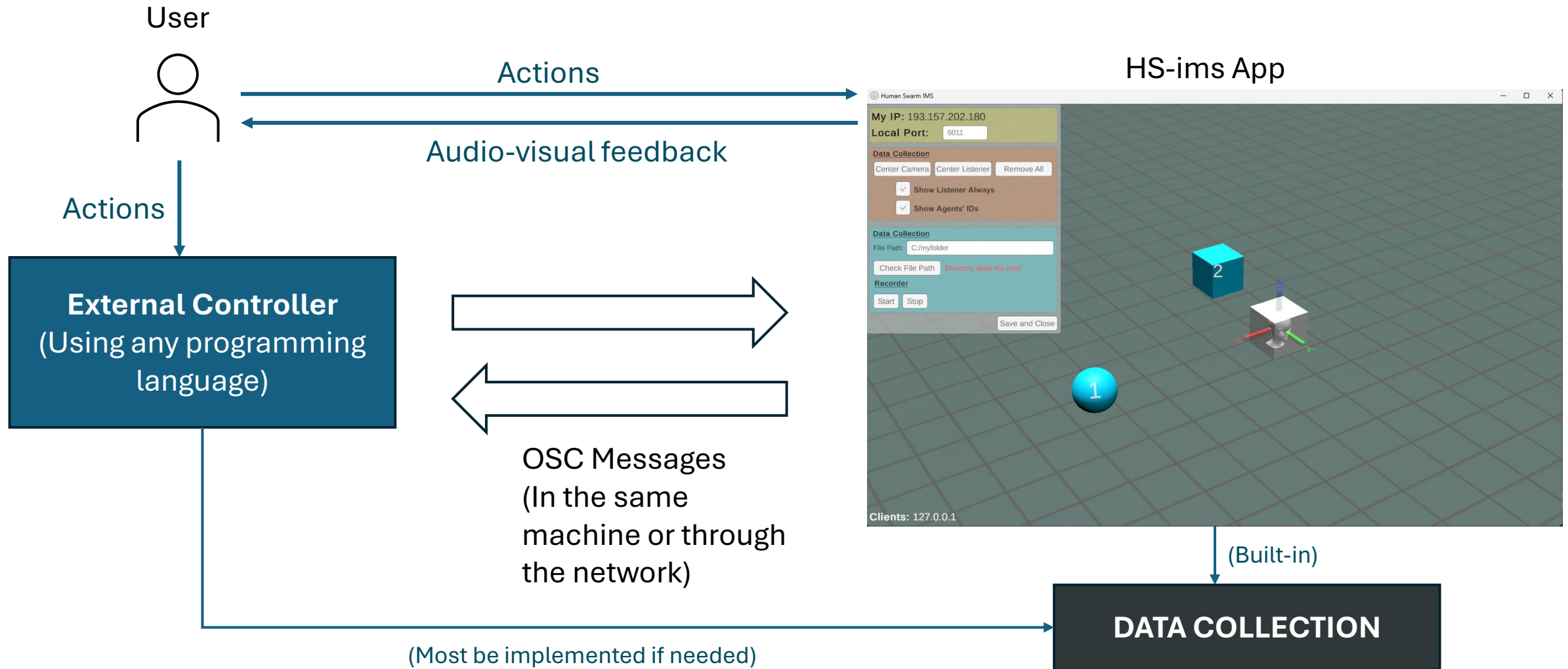
- Visualization
- Human Interaction
- Data Collection

Characteristic feature:  
**Built-in Sound Synthesis**





# HS-ims app: How does it work?



# OSC Messages

<https://ccrma.stanford.edu/groups/osc/index.html>

**OSC (Open Sound Control)** messages are packets of data used for communication between computers, audio devices, and multimedia applications.

OSC messages are typically sent over **UDP (User Datagram Protocol)**, which allows fast, efficient communication but does not **guarantee delivery**.

## Structure:

Starts with a **"/address"** (like a URL indicating the type of data).  
Followed by arguments (data values) that provide the specific information.

**Example:** `/agents/position/id 10 200 3000 5000`

Here, **"/agents/position/id"** is the address, and **"10 200 3000 5000"** the arguments representing: the agent id, position in X, position in Y, and position in Z in millimeters. An external system is telling the agent with id=10, teleport to position (200, 3000, 5000).

# OSC Bundles

OSC bundles allow multiple OSC messages to be grouped and sent together in a single transmission. Ensures that related messages are delivered together and executed in the specified order, which is crucial for synchronized control.

Like OSC messages, OSC bundles are typically sent using UDP, ensuring quick transmission.

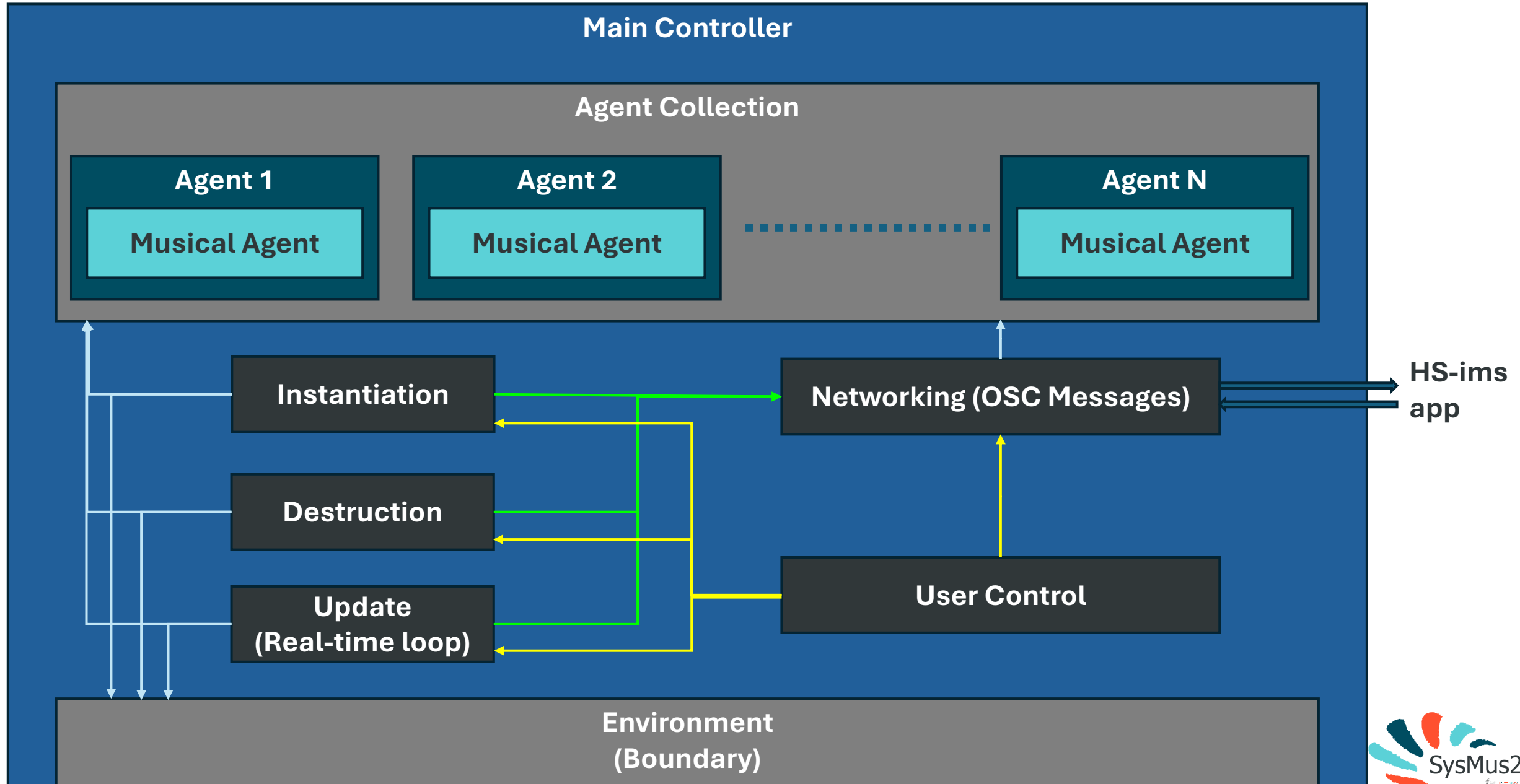
## Structure:

- Begins with "**#bundle**" identifier and a **timestamp** (indicates when messages should be executed).
- Contains one or more OSC messages in the bundle.

*The library managing OSC communication typically allows for more intuitive bundle construction.*

# IMPLEMENTATION EXAMPLE IN PYTHON

# Controller: Main Modules





# Agent Behaviour

## Movement Dynamics

$$\vec{P}(t + 1) = \vec{P}(t) + \alpha s \hat{d} \Delta t$$

$$\vec{v}(t + 1) = \frac{\vec{P}(t+1) - \vec{P}(t)}{\Delta t} = \alpha s \hat{d}$$

$\vec{P}$  : Agent position

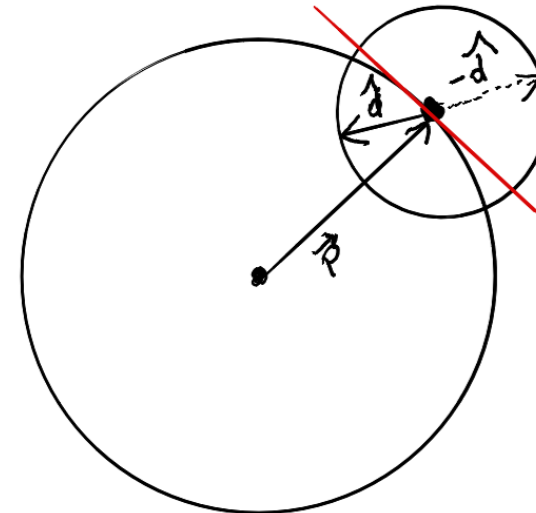
$\vec{v}$  : Agent velocity

$\hat{d}$  : Agent direction

$\alpha$ : Speed factor (modifiable)

$s$ : Speed

When the boundary limit is detected,  $\hat{d}$  changes to a random direction within the spherical environment.



# Musical Agent: Sound and Music Mapping

## Movement Dynamics

## Audio

$S = \|\vec{v}\|$  : Speed

Reverb Amount

$\frac{\|\vec{P}\|}{Br}$  : Normalized distance to the origing regarding the boundary radius(Br)

Note from a musical scale

When and while encountering nearby agents

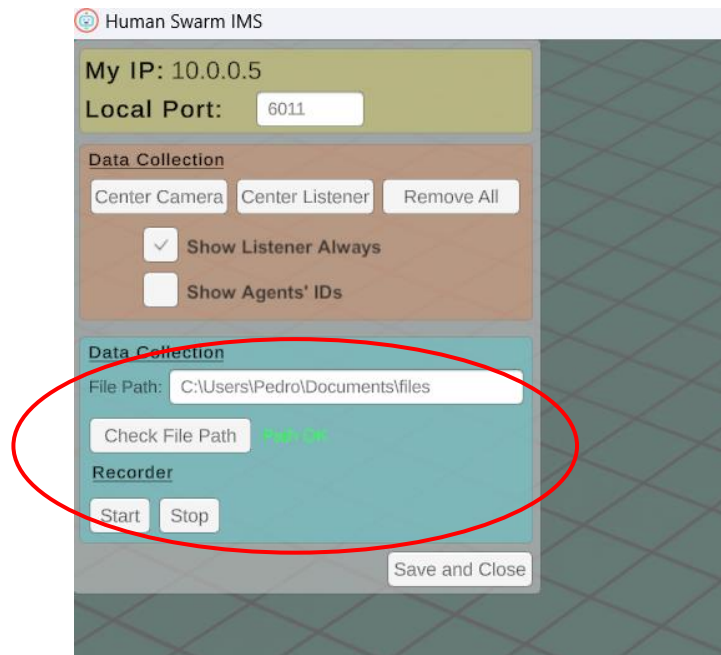
“Playing back the notes.”

# DATA ANALYSIS

# Data Collection

The app can record incoming and outgoing OSC messages in timestamped files in two ways:

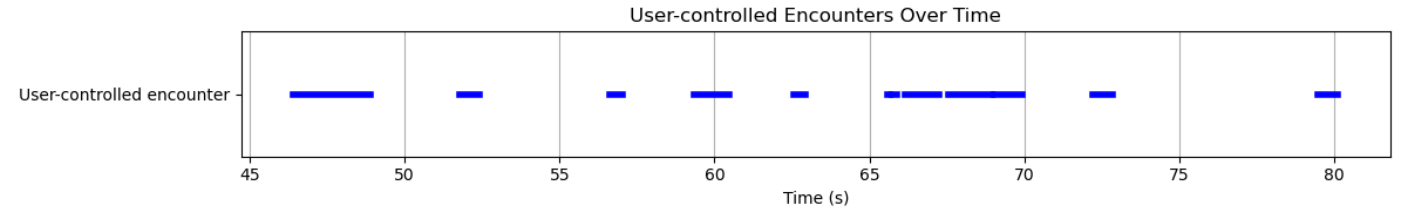
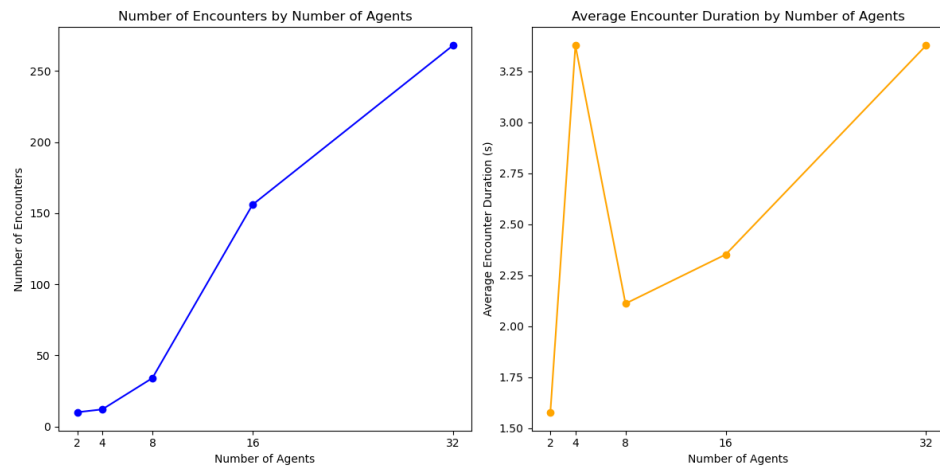
## From the App (Built-in recording control)



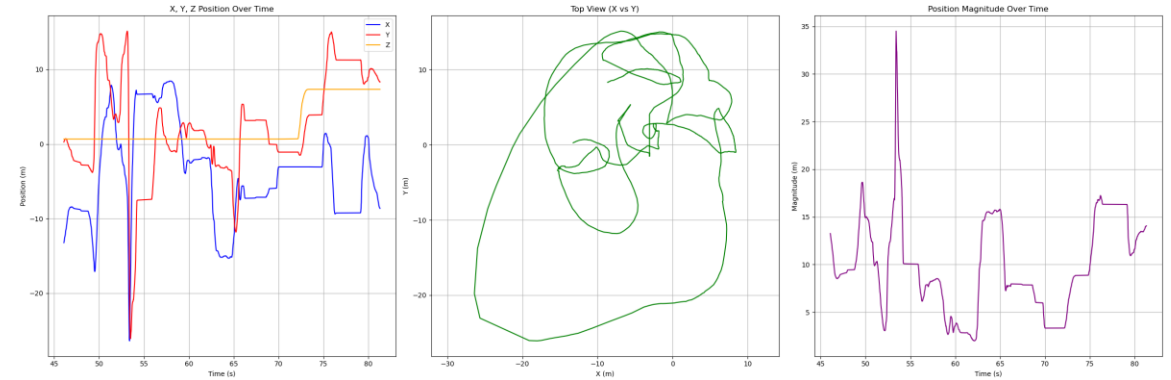
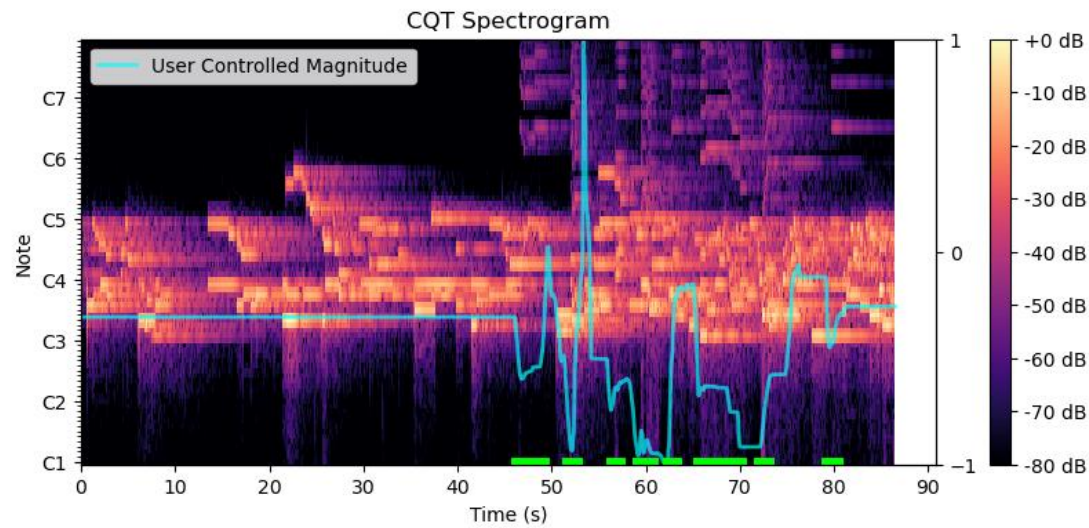
## Using OSC messages (External recording controls)

| Description     | Address         | Parameters |
|-----------------|-----------------|------------|
| Start Recording | /recorder/start | file_path  |
| Stop Recording  | /recorder/stop  |            |
| Request Time    | /time           |            |

- For data generated in the External Controller, you can implement a collection module separately.
- For audio and video recording, you can use a screen recording tool (E.g. OBS)



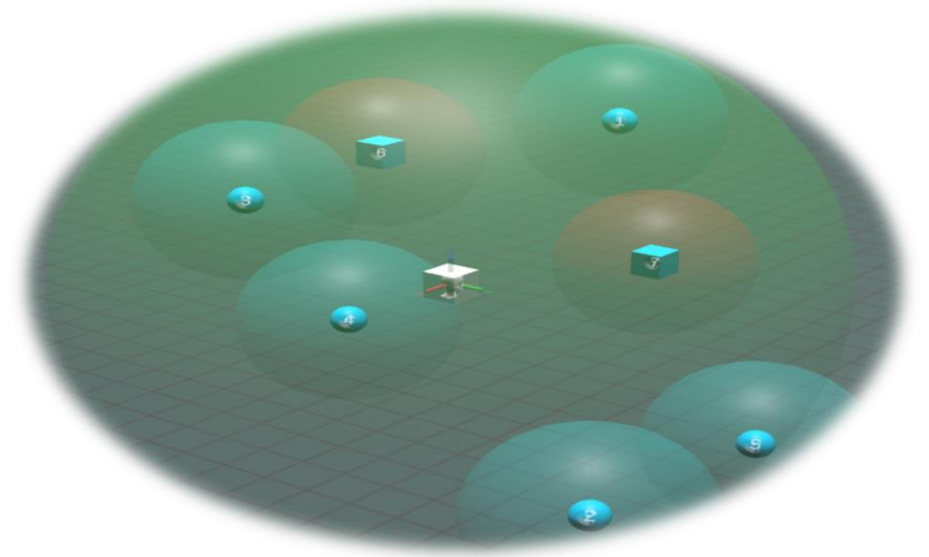
# Data Analysis for Example 1



# Human-Swarm Interactive Music Systems

Implementation and data analysis using  
Python

By *Pedro Lucas*



Post-workshop  
Form



## Questions?



<https://nettskjema.no/a/531004>